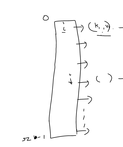

Key → HF → Gives an index b/w 0 to size-1

(K1, V1)
(K2, V2)
(K3, V3)
(K4, V4)

K1, K2, K3, K4 → HF → i

i

(K3, V3)

		Avg case.	Worst case.
Unordered Hashmaps	Insert:	O(1)	O(1)
	Search:	O(1)	O(n)
	Delete:	O(1)	O(n).

		Avg.	Worst.
Ordered Hashmaps	Insert	O(log n)	O(log n)
	Search	O(log n)	O(log n)
	Delete	O(log n)	O(log n)

Subarray with 0 sum.

[4, 2, -3, 1, 6]

i: 0 → n-1
sum = 0
j: i → n-1
sum += arr[j]
if sum == 0
return true.

TC: O(n²)
AS: O(1)

arr: [4, 2, -3, 1, 6]
PrefSum: [4, 6, 3, 4, 10]

[4, 2, -3, 1, 6].

sum = 4 2 3 4

Unordered Set.
4
6
3

TC: O(n)
AS: O(n).

[2, -3, 1, 4, 6].
0 2 -1 0 4 10

TODO: Subarray with target Sum.

[9, 5, -1, 6, 3, 2]. target = 8

9 14 13 19 22 24
8
22

[9, 5, -1, 6, 3, 2]

sum = 9 14 13 19 22

tar = 8.

Set.
0
9
14
13
19

TODO. Subarray with 0 sum.

0 1 2 3 4
[4, 2, -3, 1, 6].

sum = 4 6 3 4

Hash Map.
sum → index
0 → {-1 . . .}
4 → 0
6 → 1
3 → 2

res: [1, 3]

TODO: Subarray with tar sum.

Count subarrays with equal 0s & 1s

0 1 2 3 4 5 6
[1, 0, 0, 1, 0, 1, 1]

(0,1) (0,3) (0,5)
(1,4)
(2,3) (2,5)
(3,4)
(4,5)

res = 8.

i: 0 → n-1
count 0 = 0
count 1 = 0
j: i → n-1
if arr[j] == 0
count 0 ++
else:
count 1 ++
if count0 == count1
res ++

TC: O(n²)
AS: O(1).

-1 0 1 2 3 4 5 6
[1, -1, -1, 1, -1, 1, 1]
0 1 0 -1 0 -1 0 1
sum

(0, 5)
(2, 5)
(4, 5)

sum = 0

-1 1 -1 -1 1 -1 1 1
[1, -1, -1, 1, -1, 1, 1]

Hash Map.
sum → freq
0 → 1
1 → 2
-1 → 2

sum = 0 1 2 3 4 5 6 7

res = 1+2+1+3+1 = 8

TC: O(n).
AS: O(n).

TODO: Count the subarrays with sum = target

[0, 1, 1, 0, 0, 1, 1] x=2.

Longest Substring with Distinct Chars.

0 1 2 3 4 5 6 7 8 9 10 11
s = geeksforgeeks

res: 7.

g e e k s f o r g e e k s

res = 7

g
e
k
s

→ e f g
k o
s r